

RoMe: Row Granularity Access Memory System for Large Language Models

Hwayong Nam^{†*}, Seungmin Baek^{†*}, Jumin Kim[†], Michael Jaemin Kim[‡], and Jung Ho Ahn[†]

Seoul National University[†], Meta[‡]

[†]{nhy4916, qortmdalss, tkfkaskan1, gajh}@snu.ac.kr, [‡]michael604@meta.com

Abstract—Modern HBM-based memory systems have evolved over generations while retaining cache line granularity accesses. Preserving this fine granularity necessitated the introduction of bank groups and pseudo channels. These structures expand timing parameters and control overhead, significantly increasing memory controller scheduling complexity. Large language models (LLMs) now dominate deep learning workloads, streaming contiguous data blocks ranging from several kilobytes to megabytes per operation. In a conventional HBM-based memory system, these transfers are fragmented into hundreds of 32B cache line transactions. This forces the memory controller to employ unnecessarily intricate scheduling, leading to growing inefficiency.

To address this problem, we propose RoMe. RoMe accesses DRAM at row granularity and removes columns, bank groups, and pseudo channels from the memory interface. This design simplifies memory scheduling, thereby requiring fewer pins per channel. The freed pins are aggregated to form additional channels, increasing overall bandwidth by 12.5% with minimal extra pins. RoMe demonstrates how memory scheduling logic can be significantly simplified for representative LLM workloads, and presents an alternative approach for next-generation HBM-based memory systems achieving increased bandwidth with minimal hardware overhead.

I. INTRODUCTION

High bandwidth memory (HBM) has emerged as a key component of high-performance computing systems [28], [46]–[48] driving the transformer-based artificial intelligence (AI) proliferation [70]. The high bandwidth of HBM is required to keep pace with the compute capabilities of GPUs [48], [63], TPUs [16], [28], and other AI accelerators [14], [30], and satisfy the bandwidth-bound nature of the generation stages of the transformers [19], [55], [77]. A single cube of HBM4 [27] feeds 64 pseudo channels, each of which has 32 8 Gbps data pins, constituting a total of 2 TB/s bandwidth.

Saturating the immense HBM channel bandwidth requires efficient row-buffer utilization. A DRAM bank is composed of a 2-dimensional array of DRAM cells and indexed by row and column addresses. DRAM cell access latency is slow as it requires preceding precharge (PRE) and activation (ACT). This latency is too high to saturate such a huge channel bandwidth. To amortize those overheads, the bank prefetches an entire row into the row-buffer, which is orders of magnitude wider than the conventional DRAM access granularity. This strategy is highly effective in saturating the DRAM channel bandwidth when there exists a large spatial locality in the memory access pattern (e.g., streaming access). For example, when a series

of memory reads (RDs) target the same row-buffer, the processor-side memory controller (MC) can issue back-to-back RDs to saturate the channel bandwidth without the bubbles induced by ACT/PRE.

Instead of issuing multiple consecutive read commands, we ask: why can't DRAM access granularity simply increase to match the row-level granularity in this scenario? However, memory access patterns vary, and increasing the minimum DRAM access granularity can cause *overfetch problem* by reading unnecessary data, degrading effective bandwidth [59]. Thus, conventional DRAM access granularity is set to match the cache line size of a processor (e.g., 32 B or 64 B).

Such a cache-line-sized access granularity complicates the memory controller (MC) architecture. To support column-level accesses (i.e., RDs/WRs), an MC must maintain bank states and timing parameters. Moreover, it must operate many banks in parallel to hide the latency overhead of ACT and PRE commands [3], [41], [60]. Further complexity arises from the need to determine when to issue PRE after ACT based on access patterns (i.e., the page policy [20], [60]).

The HBM hierarchy also becomes increasingly complex due to the fine-grained access granularity. While HBM bandwidth has improved steadily [8], [27], [33], [56], the bandwidth per bank has remained nearly unchanged. As each bank operates at the DRAM core frequency and transfers data at a fixed access granularity, memory bandwidth can fundamentally be increased in only two ways: 1) by enlarging the access granularity or 2) by increasing the DRAM core frequency. However, the former is constrained by cache line size, and physical limitations prevent significantly increasing the latter.

To overcome these limits, additional hierarchical structures, such as bank group and pseudo channel (PC), were introduced to boost bandwidth [23], [24]. Bank groups combine multiple banks and deliver data to the I/O at a higher frequency, allowing transfers from different bank groups to overlap while maintaining cache line granularity. PCs increase the number of channels by reducing each channel's width, improving bandwidth without increasing both DRAM core frequency and access granularity. Unlike previous generations, HBM4 doubles total bandwidth primarily by doubling I/Os (and thus PCs) without modifying the per-channel width. While effective in raising bandwidth, these mechanisms add significant complexity to memory controller scheduling and timing.

We challenge the need for such a conventional DRAM interface paradigm in the era of transformer-based Large

* Both authors contributed equally to the paper.

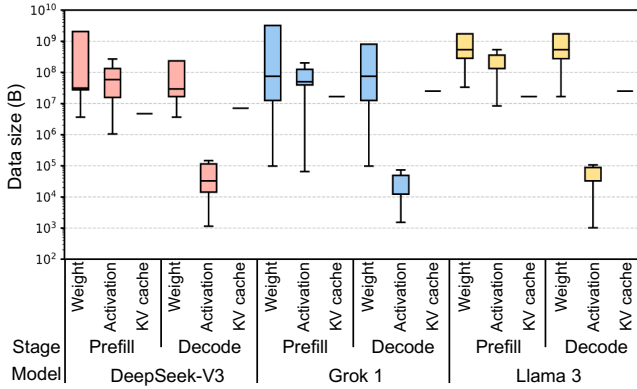


Fig. 1. Distribution of weight, activation, and KV cache size of DeepSeek-V3 [12], Grok 1 [73], and Llama 3 [13] in the prefill and decode stages.

Language Models (LLMs). While traditional systems and data centers will persist, warehouse-scale systems running homogeneous applications of LLM inference (*e.g.*, an AI factory [49]) are becoming widespread. LLMs mostly consist of simple general matrix-matrix or matrix-vector multiplication (GEMM/GEMV) and element-wise operations [55]. This is true even with state-of-the-art architectures, such as multi-head latent attention (MLA), grouped query attention (GQA), or mixture-of-experts (MoE) [37], [68], [73]. These models typically access tens of megabytes of data at once, far exceeding the size of conventional cache lines (Figure 1). Unlike the workloads with irregular or strided patterns, LLM operations exhibit highly sequential memory access patterns.

By exploiting the memory access patterns of LLMs, we propose RoMe, a **Row-granularity-access Memory** system designed to offer a simple and scalable memory system for LLM serving. First, RoMe replaces a column-level access interface with a row-level one. The sequence of DRAM commands required to access all data stored in a row is simplified into two commands: RD_row and WR_row. Second, we propose a new bank architecture, virtual bank (VBA), to further simplify the interface by removing the bank group and PC in the MC-DRAM interface. Given that memory accesses now operate at the row granularity, there is no longer a need to expose bank group and PC to the MC. Accordingly, we design a single VBA to achieve maximum bandwidth without requiring any modification to the internal DRAM structure (§IV).

Third, we introduce a command generator that decomposes each row-level command into a predefined sequence of conventional DRAM commands. This enables the integration of additional HBM channels, improving overall memory bandwidth. Finally, we demonstrate that the MC can be significantly simplified (§V-A). While conventional MCs rely on numerous data structures related to bank states and scheduling mechanisms to efficiently manage column-level commands, RoMe enables a significantly simpler and more scalable memory system design.

In a RoMe MC, five components are simplified: bank state, timing parameter, the number of bank finite-state machines (FSMs), request queue size, and scheduling algorithm. The

RoMe MC maintains only three bank states and fewer timing parameters, as the DRAM row-access command sequence is internally handled by the command generator that manages conventional timing parameters. Two or fewer VBAs operate and up to three undergo refresh simultaneously; thus, just five bank FSMs need to be maintained. This simplification enables a smaller request queue, as fewer in-flight requests need to be tracked. Finally, as one VBA can provide maximum bandwidth, the scheduling algorithm is greatly simplified, focusing solely on interleaving across VBAs.

We evaluate RoMe using three representative LLMs: Grok 1 [73], DeepSeek-V3 [12], and Llama 3 [13]. When serving LLM workloads, RoMe delivers higher performance than conventional HBM-based memory systems with significantly lower hardware overhead, while also providing modest gains in energy efficiency. This demonstrates that, under the sequential and bulk access characteristics of LLM workloads, adopting row-level memory access does not degrade performance. While minor overheads may arise from overfetch and load imbalance, their impact is negligible.

The key contributions of this paper are as follows:

- We leverage the sequential and bulky memory access patterns of LLMs to propose a memory interface based on row-level access granularity.
- As the cache-line-sized access granularity is no longer mandatory, we introduce a new bank architecture called *virtual bank*, which eliminates the need for bank groups and pseudo channels.
- A simplified memory controller optimized for the RoMe interface is designed to minimize control overhead associated with complex bank state tracking and scheduling, thereby reducing the area of the scheduling logic.
- RoMe presents a method for expanding memory channels with minimal hardware overhead, thereby improving the performance of LLM workloads.

II. CONVENTIONAL MEMORY SYSTEMS

A. Cache-Line-Sized DRAM Access Granularity

Main-memory technologies such as DDR5 [26] and HBM4, commonly integrated into modern CPUs and GPUs, are designed with access granularities that align with or are smaller than processor cache line sizes. Specifically, HBM4 is optimized for 32B accesses, aligning with the cache line size of GPUs, while DDR5 supports 64B accesses, consistent with CPU cache line sizes. Although DRAM rows are several kilobytes in size, these architectures enable fine-grained access at the column level, significantly smaller than the row size.

The adoption of cache-line-sized access granularity in main-memory systems serves two primary purposes. First, aligning the access granularity with the processor's cache line size minimizes data overfetch, thereby reducing unnecessary bandwidth usage and energy consumption by transferring only the data required by the program it executes. Second, it enables flexibility in handling diverse memory access patterns. This design effectively supports both sequential access patterns with

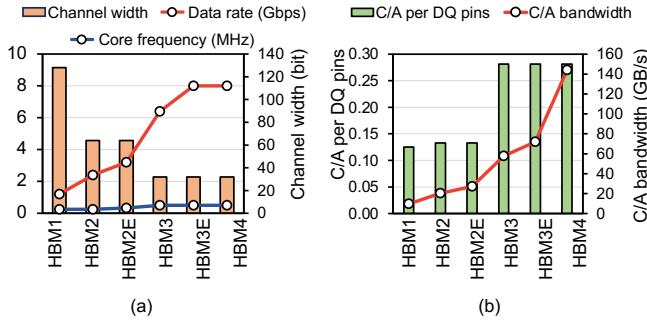


Fig. 2. (a) Trends in data rate, core frequency, and channel width, and (b) growth of C/A pin overhead across HBM generations.

high spatial locality, such as those LLMs, and random access patterns characterized by low spatial locality.

However, accessing memory at the cache line granularity introduces significant complexity in memory system design. To achieve high performance, memory controllers (MCs) should implement sophisticated scheduling algorithms that account for a wide range of timing parameters and dynamic bank states. Prior works have explored key components of this design space—including address mapping [82], page policies [20], [29], and scheduling policies [31], [32], [40], [41], [44], [60]—which further contribute to the complexity of MC architecture.

B. Bank Group & Pseudo Channel

Maintaining cache-line-sized access granularity adds complexity to the DRAM hierarchy, which in turn further increases the scheduling burden on the MC. While DRAM bandwidth has steadily improved, maintaining fine-grained cache line access necessitated additional internal structures, specifically bank groups and pseudo channels (PCs). As shown in Figure 2(a), although the external data rate of DRAM devices has consistently increased, the DRAM core frequency has shown modest growth. This limited scalability of core frequency is primarily due to the high energy and area overheads associated with its increase [51]. To meet the data rate demands under these constraints, a conventional approach has been to increase the amount of data fetched internally in a single access.

However, the increase in data rate leads to a mismatch between the access granularity and cache line size, prompting the introduction of the bank group structure [23]. Instead of doubling the amount of data fetched from a single bank (AG_{bank}), bank groups enable bandwidth scaling by alternating data fetches from banks in different bank groups at intervals of t_{CCDS} (typically equal to $t_{CCDL}/2$), while preserving the cache-line-sized access granularity (AG_{MC}). This access strategy is referred to as bank group interleaving. Each bank continues to operate at the DRAM core frequency (defined by t_{CCDL}) and fetches data at the cache line granularity. Thus, this mechanism allows effective scaling of the DRAM data rate without increasing AG_{bank} or AG_{MC} . Terminologies are summarized in Table I.

Despite the introduction of the bank group structure, the demand for even higher external bandwidth has persisted,

TABLE I
SYMBOLS AND TERMINOLOGIES

Symbol	Description
AG_{bank}	Access granularity of a bank.
AG_{MC}	Access granularity of a memory controller.

which led to the evolution of HBM toward narrower and more channels. Figure 2(a) illustrates this trend, showing a decrease in channel width and a corresponding increase in the number of channels across successive HBM generations [22], [24], [25]. In particular, each new generation of HBM has halved the channel width while doubling the number of channels. As the data rate increases, the bandwidth per channel remains constant even with the narrower channel. Notably, HBM4 scales the bandwidth by doubling the number of channels—therefore doubling the external I/O—without altering the channel width [27]. This approach enables bandwidth scaling by populating more channels while maintaining per-channel bandwidth and preserving AG_{bank} and AG_{MC} .

However, these additional hierarchies exacerbate the scheduling complexity. To fully utilize DRAM bandwidth, an MC must issue memory requests to different bank groups (*i.e.*, bank group interleaving) and PCs. This requires the MC to continuously track the state of all banks to identify those that are ready to accept new DRAM commands. As a result, the MC must employ more sophisticated scheduling mechanisms to effectively leverage the complex DRAM hierarchy.

As the channel width narrows with each HBM generation, the overhead associated with command/address (C/A) pins increases. HBM defines separate pins for row and column commands; for example, in HBM4, each 64-bit data channel requires 10 row command pins and 8 column command pins. Moreover, populating more PCs proportionally increases independent C/A pins, raising the C/A-to-DQ pin ratio (see Figure 2(b)). From HBM1 and HBM2/2E to HBM3/3E and HBM4, this ratio has nearly doubled. Further, the bandwidth requirements of C/A pins have steadily increased across generations, contributing to the rising overhead of the C/A interface. Adopting these same techniques for future HBM generations with higher pin rates and bandwidths may be unsustainable.

C. HBM Architecture

HBM stacks multiple DRAM dies with a logic die at the bottom, which are connected by through silicon vias (TSVs), as shown in Figure 3. Each HBM device is composed of multiple channels—up to 32 channels in the case of HBM4 [27]—and forms a Stack ID (SID, equivalent to rank in conventional DRAM standards) for every four DRAM dies, supporting up to four SIDs per device. Each channel uses the SID to identify which group of DRAM dies it is accessing. Each channel consists of two PCs, a design unique to HBM. Two PCs in each channel share C/A pins but split the data pins evenly. The two PCs can operate independently, enabling concurrent data transfers and maximizing throughput.

Data transfer from individual banks within the DRAM dies to the logic die occurs as follows. Each bank fetches 256 bits

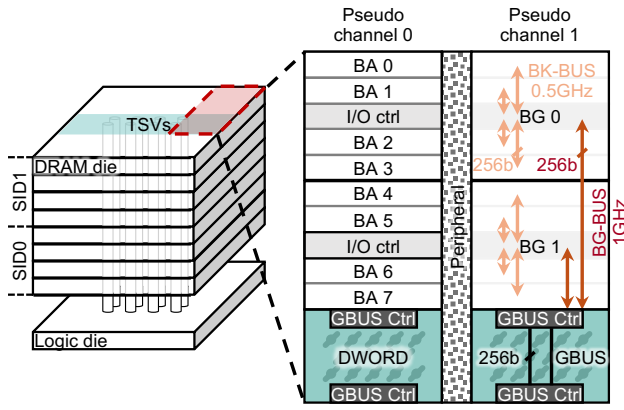


Fig. 3. Overview of HBM architecture and internal organization.

of data, corresponding to AG_{bank} , and delivers it to the I/O control (ctrl) buffer via the bank data bus (BK-BUS). Since all banks within a bank group share a single I/O control buffer, only one bank can occupy the BK-BUS at a time. The data stored in the I/O ctrl buffer is then transferred over the bank group data bus (BG-BUS) to the global data bus (GBUS) controller and ultimately delivered to the logic die via the TSVs. BK-BUS runs at the frequency of $1/t_{CCDL}$ (e.g., 0.5 GHz), whereas BG-BUS runs at a faster frequency of $1/t_{CCDS}$ (e.g., 1 GHz). Therefore, a single bank group can utilize only half of the available bandwidth. To fully exploit the maximum bandwidth, data must be transmitted in a time-multiplexed manner across different bank groups.

D. Conventional Memory Controller Architecture

While implementation details vary, the high-level architecture of a generic MC is depicted in Figure 4. MC generally includes four core components: address mapping, read/write request queue, per-bank state logic, and a command scheduler. The address mapping unit translates the physical address of each read/write request received from the host into a corresponding DRAM address [6], [21], [42], [58], [71], [72] (e.g., PC and bank group) and inserts the translated request into the request queue. Both the request queue and bank state logic are commonly implemented using content-addressable memory (CAM), allowing a one-cycle lookup to identify ready requests [5]. High bandwidth utilization requires a sufficiently large CAM to accommodate numerous in-flight requests. As banks operate independently, per-bank state logic tracks the status of each bank. The command scheduler is responsible for issuing memory and refresh commands by evaluating all bank states while adhering to DRAM timing constraints. Each bank can be in one of seven states: Idle, Activating, Active, Precharging, Reading, Writing, and Refreshing. The command scheduler must manage a wide range of timing parameters, which are summarized in Table II.

Although the command scheduler performs various tasks, its responsibilities can be broadly categorized into refresh and request scheduling. Refresh scheduler periodically issues refresh (REF) commands according to the t_{REFI} interval,

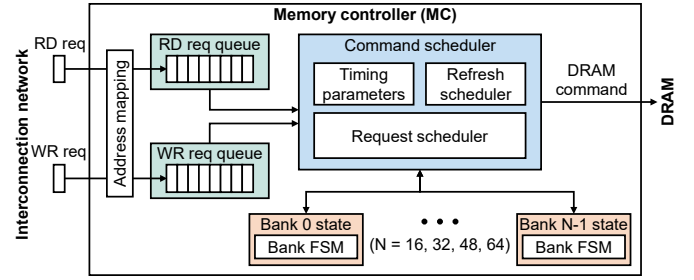


Fig. 4. Conventional memory controller architecture.

TABLE II
SUMMARY OF HBM TIMING PARAMETERS

Parameter	Description
tRCDRD	ACT to RD delay in a same bank
tRCDWR	ACT to WR delay in a same bank
tRAS	ACT to PRE delay in a same bank
tRP	PRE to ACT delay in a same bank
tCCDS(L/R)	RD/WR to RD/WR delay in diff BG (same BG/diff rank)
tFAW	Time window for 4 ACTs
tRRDS(L)	ACT to ACT delay in diff/same BG
tWTRS(L)	WR to RD delay in diff/same BG
tRTW	RD to WR delay in a same bank
tWR	WR to PRE delay in a same bank
tRTP	RD to PRE delay in a same bank

while optionally postponing or pooling REFs based on each bank's state [27].

Request scheduler determines which request to schedule based on multiple criteria. First, it exploits interleaving across banks, bank groups, and PCs. Bank interleaving helps hide ACT and PRE latencies by overlapping operations across independent banks. Interleaving across bank groups and PCs further increases bandwidth utilization. Second, the scheduler aims to exploit row buffer locality by issuing as many RDs/WRs as possible to an open row while obeying fairness; it pursues confining the overhead associated with ACT and PRE. Third, it manages the page policy by determining the optimal time to precharge a row after activation, depending on memory access patterns. This policy balances latency with row buffer hit rate and is typically implemented using open, close, or adaptive page policies [29], [50]. Finally, to prevent starvation caused by the aggressive scheduling strategies, the scheduler incorporates Quality-of-Service (QoS [10]) mechanisms that prioritize long-waiting requests, ensuring fairness across all memory transactions [40], [41].

III. ACCESS PATTERN OF LARGE LANGUAGE MODELS

Widely adopted large language models (LLMs) are typically built upon the transformer decoder architecture (see Figure 5). Throughout this paper, the term LLM refers specifically to a transformer-based LLM. LLM inference can be broadly divided into two stages: `prefill` and `decode`. In the `prefill` stage, the model ingests all input tokens (e.g., words) in the request and generates the first output token. In the `decode` stage, it operates auto-regressively, taking the

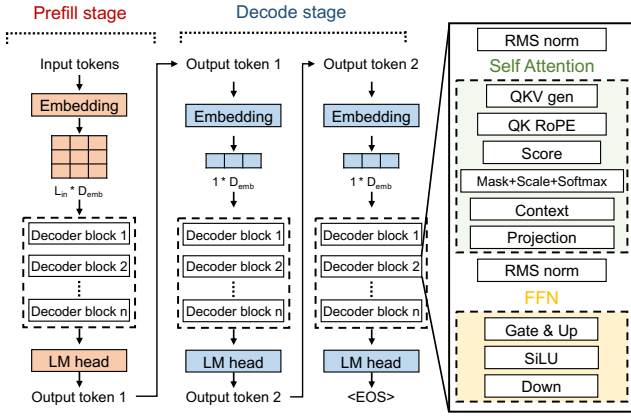


Fig. 5. Transformer-based LLM architecture.

output token from the previous step as input to produce the next output token.

Each stage consists of a token embedding layer, multiple decoder blocks, and a language model (LM) head. When an LLM processes an inference request containing a sequence of tokens, the embedding layer maps the input tokens into hidden vectors, which are then passed through the decoder blocks. Each decoder block takes the hidden vectors from the preceding decoder block and produces updated hidden vectors. Finally, the hidden vectors from the last decoder block are transformed into tokens by the LM head. Here, we refer to the pre-trained model parameters (e.g., the weight of the fully-connected layer) as weight and the intermediate results of the operations and layers as activation.

In addition to the weights and activations, LLMs have a third primary data type: the KV-cache. Each decoder block is mainly composed of a self-attention layer (attention) and a feed-forward network (FFN). The attention layer takes the hidden vector as input and produces the Query (Q), Key (K), and Value (V) matrices. Because K and V store sequence context, the model requires K and V matrices for the entire sequence to generate each new token. To avoid repeating the same computation at every generation step, the K and V matrices are stored in the KV-cache. Thus, the data used in LLM computation can be broadly categorized into weights, activations, and the KV-cache.

During LLM execution, tens of megabytes of data typically need to be accessed sequentially at a time. For all three LLMs in Figure 1, most weight and KV-cache accesses exceed several hundred kilobytes. In Grok-1, only one weight matrix is exceptionally small (24 KB), but all other weight matrices exceed 12 MB. The KV-cache also reaches several megabytes in the decode stage; it grows even larger than in the prefill stage because it must hold KV-cache for both the input and the already generated output tokens. For activations, the prefill stage processes all input tokens as a single batch, resulting in activation sizes reaching tens of megabytes. In the decode stage, however, only one token per sequence is processed, so the activation size is much smaller. Nevertheless, given that modern LLM services often run with batch sizes in

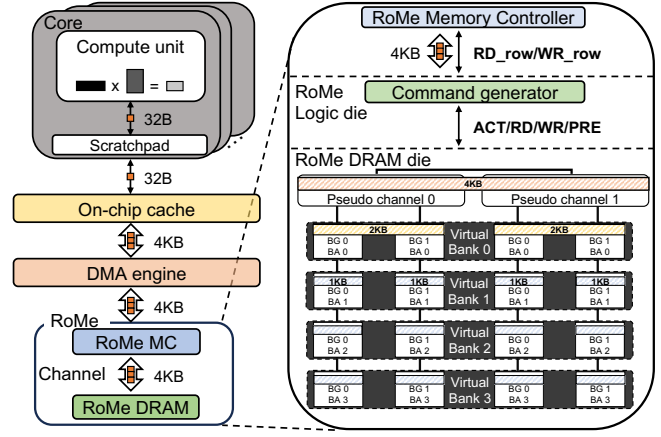


Fig. 6. An overview of a RoMe-based system.

the hundreds [19], [55], [57], [76], [77], the activations can scale to a few megabytes, similar to the weights.

As GEMM and GEMV operations dominate LLM computations, these data are accessed with simple sequential memory access patterns. However, current HBM-based memory systems are still designed for extremely fine-grained 32 B accesses, introducing unnecessary complexity relative to access characteristics of LLMs. Therefore, we propose a highly simplified memory interface optimized for the sequential access pattern of LLMs and provide an in-depth analysis of its benefits. We then present a co-optimization of DRAM and MC based on this interface, demonstrating a memory system for next-generation AI accelerators that scales more effectively.

IV. THE RoMe INTERFACE

A. Memory Interface

Exploiting the sequential and coarse-grained memory access patterns of LLM workloads, we propose a **Row** granularity access **Memory** system, RoMe (Figure 6). For systems serving LLMs that sequentially access hundreds of megabytes of data at a time, the conventional cache-line-sized access granularity is excessively fine-grained. RoMe replaces the traditional cache-line-level (column-level) interface with a row-level interface comprising RD_row and WR_row. This increases AG_{MC} from cache-line size to row size. Because AG_{MC} now corresponds to the row size, it is no longer necessary to align AG_{bank} with the cache-line size. Thus, we can further streamline the MC-DRAM interface by eliminating the bank group and PC from the interface that were originally introduced to scale bandwidth while retaining cache-line-sized AG_{bank} . With this significantly simplified interface, the RoMe MC no longer requires complex scheduling logic, leading to a much simpler architecture.

Moreover, we integrate command generators on the logic die to translate row-level commands into conventional DRAM commands. This integration reduces the C/A pin count required per channel, allowing an HBM to add more channels with only a slightly increased pin budget, providing additional aggregate bandwidth. Through both the simplified MC and the command generator, RoMe improves memory bandwidth

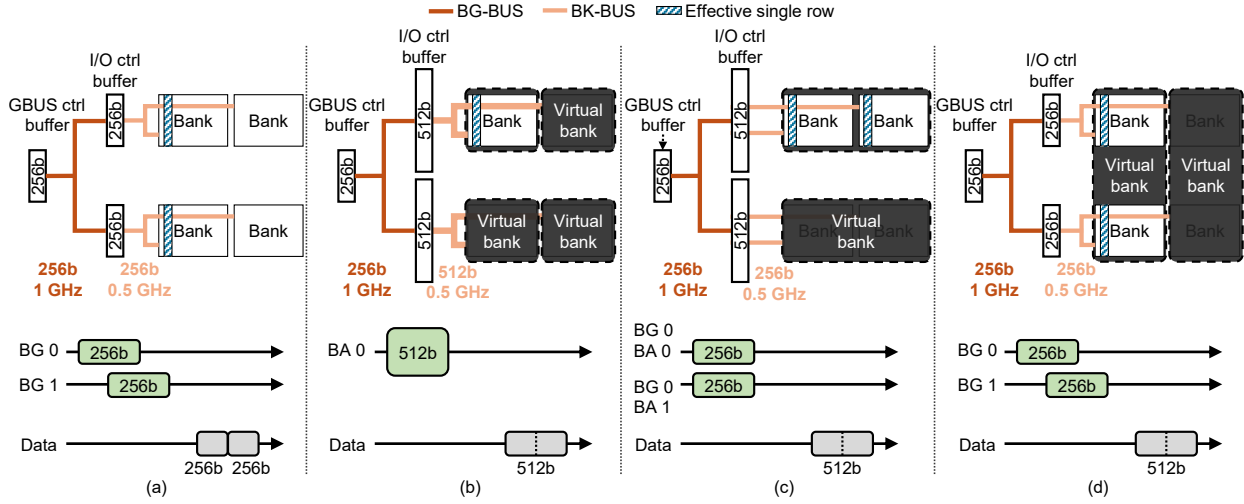


Fig. 7. Three design approaches to eliminate the bank group from the MC-DRAM interface. (a) Conventional bank group architecture. (b) A single bank serves as a VBA by doubling AG_{bank} . (c) Two banks operate in tandem to form a VBA. (d) Two banks from different bank groups form a VBA and fetch data in an interleaved manner.

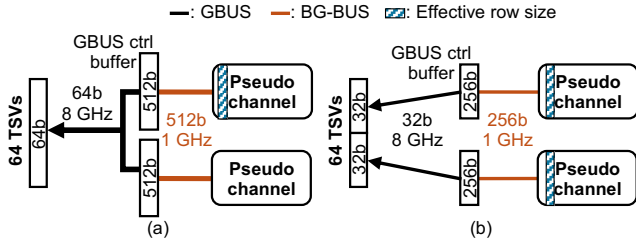


Fig. 8. Two design approaches to eliminate PC from the MC-DRAM interface. (a) A single PC operates as a full channel by fetching double the cache-line size. (b) Two PCs serve as a single channel and operate simultaneously, each maintaining data fetch size.

with low hardware overhead, demonstrating decent scalability. Their implications on area and energy are described in §VI-C.

RoMe is designed to interoperate smoothly with modern AI accelerators. LLM inference must continuously process massive weight and activation data, demanding not only high compute throughput, but also memory with both high bandwidth and high capacity. Consequently, HBM has emerged as the standard memory for AI accelerators, and RoMe is accordingly designed to utilize the latest generation, HBM4. Moreover, modern AI accelerators have adopted techniques that issue bulky memory accesses to efficiently fetch the enormous data required by AI workloads [28], [47]. In line with this trend, we assume a system where memory requests on the order of kilobytes are delivered to the MC.

B. Virtual Bank

RoMe removes the concepts of bank group and PC, replacing them with a new hierarchy, a virtual bank (VBA). The key idea behind VBA is to deliver the full available bandwidth from a single VBA, eliminating the need for complex MC-side scheduling that accounts for bank group or PC interleaving. Because row granularity access no longer requires matching AG_{bank} and AG_{MC} to the cache-line size, traditional bank group and PC interfaces are no longer essential. Accordingly,

various design choices are possible for implementing VBA, and this work seeks to analyze the trade-offs associated with each.

There are three main design spaces for implementing a VBA that achieves maximum bandwidth from a single VBA. First, as illustrated in Figure 7(b), a single bank can serve as a VBA by increasing its AG_{bank} , thereby enabling it to deliver the maximum bandwidth. While it maintains the same number of banks and effective row size, it requires doubling the bank's internal data path, the BK-BUS width, and the I/O ctrl buffer size, resulting in significant area overhead [51]. Second, as shown in Figure 7(c), a VBA consists of two banks within the same bank group. By operating two banks in tandem, this approach fetches twice the amount of data, doubling the effective AG_{bank} . Although this does not change the internal data path, BK-BUS width, and I/O ctrl buffer, it effectively reduces the total number of banks by half and doubles the effective row size. Finally, as shown in Figure 7(d), a VBA consists of two banks from different bank groups, accessed in a time-multiplexed manner. This approach leverages the existing DRAM structure without modification while still enabling a single VBA to achieve maximum bandwidth. Similar to the second design, it reduces the number of banks by half and doubles the effective row size, but does so without requiring changes to the internal DRAM architecture.

There are two design approaches for eliminating the concept of PC. Figure 8(a) illustrates a method where the amount of data fetched from each PC is doubled to enable a single PC to achieve maximum bandwidth. However, this approach necessitates an increase in BG-BUS width and the buffer size of the I/O ctrl buffer. Moreover, multiplexers are required between two GBUS on each PC side. As a result, from the MC's perspective, the two PCs are controlled as a single channel, with the effective row size remaining 1KB while doubling the number of banks. Figure 8(b) shows that both PCs operate simultaneously, similar to the legacy channel mode in HBM1/2 [22], [24]. This configuration doubles the bandwidth

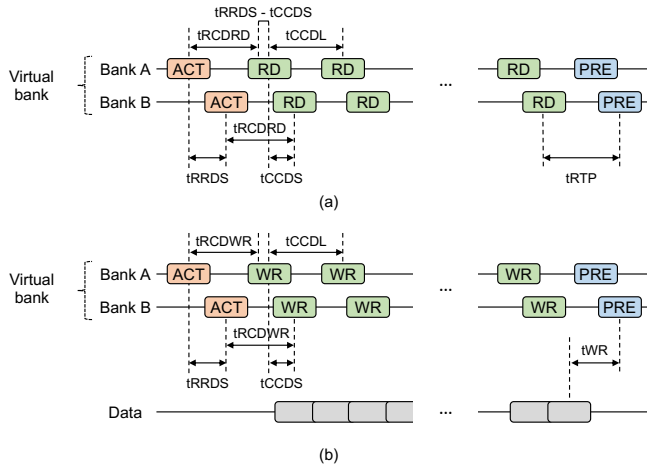


Fig. 9. Command sequence of (a) RD_row and (b) WR_row.

without requiring additional wiring and buffer, though the effective row size increases to 2 KB. In LLM workloads that involve fetching several megabytes of data, this increase in effective row size is not a significant issue. Therefore, we eliminate the PC from the MC-DRAM interface by enabling the concurrent operation of two PCs (Figure 8(b)).

We conducted a comprehensive exploration of all combinations within the VBA design space, with the methodology and workloads detailed in §VI-A. Our experiments covered six total configurations, generated by combining the design points in Figure 7(b)/(c)/(d) with those in Figure 8(a)/(b). Across all six configurations, the performance deviation relative to the baseline system remained within 3.6%.

However, the designs exhibit significant differences from the perspective of area overhead. Using the configuration shown in Figure 8(a) requires doubling the width of the BG-BUS. Similarly, the I/O ctrl buffer for Figure 7(b)/(c) must also be doubled, which in turn necessitates doubling the BK-BUS width for Figure 7(b). When combined with the design in Figure 7(b)—where the BK-BUS and internal bank datalines are already doubled—the total dataline width becomes $4\times$ that of traditional bank architecture, resulting in a substantial area overhead up to 77% [51]. Thus, we adopt the configuration in Figure 7(d) and Figure 8(b).

C. Command Generator

We add a command generator that accepts row-level commands and streams data from the VBA. When the MC issues a RD_row or WR_row command, the command generator translates it into a fixed sequence of DRAM commands: one ACT, a series of RD or WR commands, and a PRE. Unlike a conventional MC, our command generator does not issue commands dynamically based on bank states or timing constraints. Instead, it issues predetermined DRAM commands at fixed intervals upon receiving a row-level command, operating in a simplified and static manner. Figure 11 illustrates the detailed command sequences corresponding to RD_row and WR_row. In RoMe, as in the legacy channel mode of HBM1/2, commands are sent to both PCs and data are also received

from both simultaneously. Because two PCs share the same C/A pins, we depict the command sequences for a single PC.

The command generator is designed to issue DRAM commands to two banks in a perfectly interleaved manner, ensuring that each RD/WR complies with t_{CCDS} (e.g., 1 ns) between consecutive RD/WR to a different bank. However, due to the t_{RRDS} constraint (e.g., 2 ns), which must be satisfied between ACTs to different banks, maintaining this interleaving necessitates additional delay. If both banks issue ACT followed by RD after t_{RCDDR} , the RDs to different banks would align simultaneously rather than being interleaved. To resolve this, an intentional delay of $t_{RRDS} - t_{CCDS}$ is inserted before the ACT to the first bank (Figure 9). This allows the RDs/WRs to the two banks to be issued at t_{CCDS} intervals.

The command generator can be placed in one of three locations: 1) MC, 2) logic die, or 3) DRAM die. Placing the command generator in the MC has the benefit of minimizing modifications to the existing memory system. However, this configuration limits the structural advantages that can be gained from a simplified memory interface, such as reducing C/A pins. Integrating the command generator within the HBM stack helps reduce the C/A pin count between the MC and HBM. When placed in the logic die, the command generator can reduce the C/A pin count between the MC and the logic die, though it does not reduce the number of TSVs between the logic and DRAM dies. Placing it in the DRAM die can reduce TSV usage between the logic and DRAM dies, but it requires one command generator per channel for each DRAM die, increasing redundancy.

Given these trade-offs, we adopt a middle-ground design by placing the command generator in the logic die. First, because the logic die of HBM4 is fabricated using a logic process (rather than a DRAM process) [66], [67], placing one command generator per channel incurs minimal area overhead (quantified in §VI-C) while enabling effective reduction in C/A pin count. Second, recent advances in die-stacking technologies, such as hybrid bonding [15], [45], help alleviate the cost associated with inter-die TSVs, making the logic-die placement a practical compromise.

D. Command/Address Pins

Row granularity access enables a drastic reduction in the number of C/A pins between the MC and DRAM. First, because separate RD and WR column C/A pins are no longer required, eight column C/A pins can be removed. The mode register set (MRS), which is traditionally sent over a column command, is now transmitted via row C/A pins. Out of the ten row C/A pins, up to four pins are used for the opcode, leaving pins for the address. RoMe retains all four opcode pins but reduces the number of address pins. Since RoMe does not require PC bits and each VBA includes two banks, one of the bank address bits is also unnecessary. Excluding ACT and PRE, there are eight row commands. Adding MRS, RD_row, and WR_row increases the total command count to eleven. In a column-granularity interface, column C/A pins must support

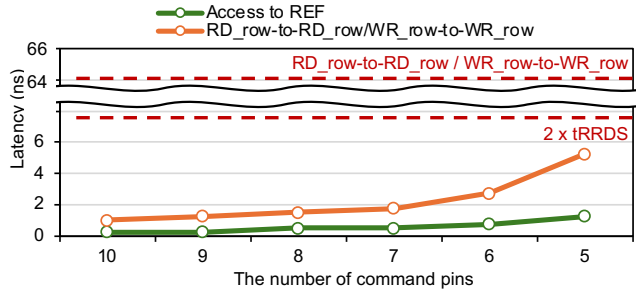


Fig. 10. Latency between RD_row/WR_row and REF across various numbers of C/A pins.

issuing RD and WR commands to both PCs every t_{CCDS} , and row C/A pins must support ACT commands every t_{RRDS} .

However, with the row-level interface, the minimum interval between commands is longer. The tightest timing occurs when a REF command is issued immediately after a RD_row or WR_row , requiring at least $2 \times t_{RRDS}$. This is because one t_{RRDS} delay is needed between the ACT commands to the first and second banks, and another t_{RRDS} delay is needed before issuing the REF command to the second bank. Figure 10 shows the command issue latency as a function of the number of C/A pins. Even with just five pins, commands can still be issued faster than $2 \times t_{RRDS}$. Therefore, by reducing the number of C/A pins to five, RoMe is able to eliminate 72 % of the C/A pins.

E. Additional Channels

We utilize the freed C/A pin margin to introduce additional channels. RoMe reduces the number of C/A pins from 18 to 5, saving 13 pins per channel. A channel of HBM4 requires 120 pins [27], whereas RoMe requires only 107 pins due to the 13-pin reduction. Consequently, in a 32-channel configuration, 416 pins remain available, which allows the addition of four new channels with only 12 extra pins. Through these additional channels, we aim to increase the memory bandwidth.

RoMe proposes to increase memory bandwidth by adding one additional channel per DRAM die. As HBM generations have evolved, the number of channels per die has increased for channel expansion, necessitating a larger die area [8], [33], [34], [52], [56]. Following this trend, RoMe also adopts a design expanding the number of channels per DRAM die from eight to nine. As a result, RoMe-based HBM achieves approximately a 12.5% increase in memory bandwidth merely with a small number of additional pins at the processor interface. The area overhead is estimated in §VI-C.

V. MEMORY SYSTEM UNDER RoMe INTERFACE

A. RoMe Memory Controller Architecture

As the memory interface is simplified, the MC can also be significantly simplified. The MC now issues only three row-level commands (RD_row , WR_row , and REF), so the timing constraints among ACT, PRE, and RD/WR typical of conventional DRAM interfaces are eliminated. Row-granularity operation also reduces the bank states in the bank FSM and timing parameters, and adopting VBA reduces the complexity

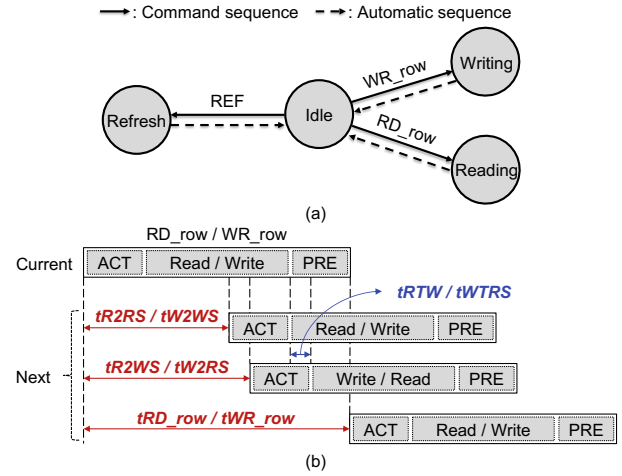


Fig. 11. (a) Bank state diagram and (b) timing parameters of RoMe MC.

TABLE III
TIMING PARAMETERS OF RoMe

Name	Description	Destination
t_{R2RS}	Different VBA	RD_row to RD_row
t_{R2RR}	Different SID	RD_row to RD_row
t_{R2WS}	Different VBA	RD_row to WR_row
t_{R2WR}	Different SID	RD_row to WR_row
t_{W2RS}	Different VBA	WR_row to RD_row
t_{W2RR}	Different SID	WR_row to RD_row
t_{W2WS}	Different VBA	WR_row to WR_row
t_{W2WR}	Different SID	WR_row to WR_row
t_{RD_row}	Same VBA	RD_row delay
t_{WR_row}	Same VBA	WR_row delay

of bank-state tracking logic. Finally, the scheduler's complexity for maximizing bandwidth is greatly reduced.

Bank states: Row-level access drastically simplifies the bank states related to data access. Figure 11(a) illustrates the bank states of RoMe, which are Idle, Writing, Reading, and Refreshing. Idle means the VBA is ready to accept a DRAM command immediately. Refreshing indicates that a REF command is in progress. Reading and Writing mean the bank is executing a RD_row or WR_row command, respectively. In conventional DRAM, after a Reading or Writing state, the bank returns to an Active state and can dynamically transition to additional reads or a precharge. Under RoMe, however, DRAM is accessed only via RD_row and WR_row commands. Upon completion, the bank automatically returns to Idle. Thus, the Active, Activating, and Precharging states are no longer needed.

Timing parameters: The RoMe MC considers only a minimal set of timing constraints when performing memory accesses. It issues only RD_row , WR_row , and REF commands, and each command returns its bank to the Idle state automatically. Thus, it must track only the timing relationships between RD_row and WR_row commands. Rather than juggling the full set of row C/A and column C/A timing parameters in a conventional interface, the MC needs to manage only a few timing parameters. Table III lists the ten timing parameters used in RoMe, categorized by Read-to-Read, Read-to-Write, Write-to-Read, and Write-to-Write for same-bank, different-bank, and different-stack-ID cases. Figure 11(b) illustrates

TABLE IV
SIMPLIFIED COMPONENTS OF RoMe MC

	Conventional MC	RoMe MC
# of timing params.	15	10
# of bank FSMs	# of total banks per PC	5
# of bank states	7	4
Page policy	Open	-
Scheduling	Row-buffer locality, Bank group interleaving PC interleaving	VBA interleaving

when each timing parameter applies. For t_{R2RS} and t_{W2WS} , the next data transfer to the same row can begin immediately after the current one finishes. For t_{R2WS} and t_{W2RS} , the bus direction must switch, so an additional t_{RTW} or t_{WTRS} delay is incurred. Accesses to different stack IDs (t_{R2RR} , t_{R2WR} , t_{W2WR} , and t_{W2RR}) incur a $1-2n_{CK}$ longer delay than different-bank accesses [27]. Finally, t_{RD_row} and t_{WR_row} simply chain within the same VBA, so the next operation can start as soon as the previous one completes.

The number of bank FSMs: Because RoMe drives at most two VBA at any given time, the MC needs only two bank FSM instances for scheduling. Since two VBAs can saturate the bandwidth, the MC needs only to track the currently accessed VBA and the next VBA. Nevertheless, due to the use of per-bank refresh, additional bank FSMs are implemented to track the status of banks being refreshed for a duration of t_{RFCpb} divided by t_{REFIpb} . Memory requests are mapped to whichever bank FSM instance is free, and once a request completes, that FSM is deallocated.

Request queue size: The RoMe MC employing a highly simplified scheduler treats each 4KB access as a single request, enabling it to saturate DRAM bandwidth with a significantly smaller request queue. In a cache-line access granularity, the ratio of t_{CCDS} to t_{RC} exceeds $40\times$, whereas with row granularity access, the ratio of t_{R2RS} to t_{RD_row} is less than $2\times$. If the queue size is too small, it cannot look far enough ahead to exploit bank-level parallelism; thus, a certain minimum size is still necessary. HBM4 requires a queue depth of at least 45 entries, while RoMe achieves peak throughput with only two entries. Thus, RoMe can saturate DRAM bandwidth with a depth of just two, allowing the MC to reduce the request queue size.

Command scheduling: The command scheduler delivers high performance and fairness with minimal complexity. It first checks which VBA is active, then serves ready requests in oldest-first order. As row-buffer locality is guaranteed by row granularity access, the scheduler needs only to avoid back-to-back commands to the same VBA to fully utilize bandwidth. An age-based mechanism ensures that the oldest pending request is served next, improving tail latency and fairness.

Row-level access removes the need for any page-policy mechanism. Conventional MCs dynamically switch between open, close, and adaptive page policies by monitoring row-buffer hits to adapt to varying access patterns [20]. In contrast, RoMe always precharges immediately after reading a row, in-

TABLE V
TIMING PARAMETERS OF HBM4 AND RoMe

	HBM4	RoMe
channels/cube (PCs/cube)	32 (64)	36
stacks	4	4
banks/channels	128	32
row size	1 KB	4 KB
data rate	8 Gb/s	8 Gb/s
bandwidth	2 TB/s	2.25 TB/s
timing parameter (ns)	$t_{RC}=45$, $t_{RP}=16$, $t_{RAS}=29$, $t_{CL}=16$, $t_{RCDRD}=t_{RCDWR}=16$, $t_{WR}=16$, $t_{FAW}=12$, $t_{CCDL}=2$, $t_{CCDS}=1$, $t_{CCDR}=2$, $t_{RRD}=2$	$t_{R2RS}/R=64/68$ $t_{R2WS}/R=69/73$ $t_{W2RS}/R=71/75$ $t_{W2WS}/R=64/68$ $t_{RD_row}=95$ $t_{WR_row}=115$
AG_{MC}	32 B	4 KB

herently matching LLM’s sequential access without requiring any additional policy logic.

B. Refresh and Write Operations

We optimize refresh behavior to suit the simplified interface with VBA. For all-bank refresh (REFab), no bank in the target channel can operate during a refresh; thus, both the baseline and the RoMe MC behave the same. By contrast, for per-bank refresh (REFpb), triggering a REF command on any single bank within a VBA blocks the entire VBA. Thus, it is important to minimize this overhead. Instead of issuing a REFpb every t_{REFIpb} , MC issues one per-bank refresh every $2\times t_{REFIpb}$. The command generator then sends two REFpb commands (one to each bank in the VBA) with an interval defined by the REFpb-to-REFpb timing interval, t_{RREFD} . This reduces the stall time per VBA from $2\times t_{RFCpb}$ (e.g., 2×280 ns) to $t_{RFCpb}+t_{RREFD}$ (e.g., 280 ns + 8 ns).

Buffering multiple 4 KB write chunks in a write queue would require a large write buffer. To avoid this, RoMe processes write requests immediately upon arrival, keeping the queue size small. Since LLM workloads are heavily dominated by reads, the impact of immediate write handling is minimal. Additionally, by issuing large 4 KB write requests atomically, RoMe reduces the frequency of read/write turnaround delays.

VI. EVALUATION

A. Methodology

System: We first describe the configuration of a single accelerator and then extend the design to a multi-accelerator system representative of real LLM deployments. Modern AI accelerators exhibit arithmetic intensities of 200–300 Op/B for BF16 operations (e.g., 281 Op/B on B200 [48]) and attach up to eight HBM cubes per device [48]. Accordingly, we configure our target accelerator to sustain 280 Op/B and connect to eight HBM4 cubes. Each HBM4 cube provides 32 GB capacity with 8 Gbps data rate and a 16-Hi configuration [27], yielding total 256 GB memory system with 16 TB/s bandwidth. To match our target arithmetic intensity, we scale BF16 throughput to 4480 TFLOPS. Because real-world LLM deployments often span multiple devices to meet high capacity demands, we evaluate a system with eight accelerators operating in parallel,

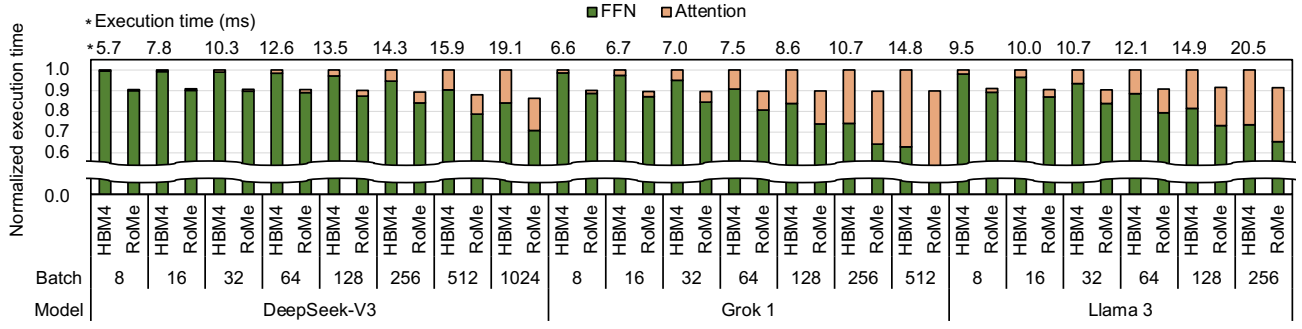


Fig. 12. TPOT (time per output token) comparison between HBM4-based memory system and RoMe across various batch sizes for DeepSeek-V3, Grok 1, and Llama 3. The sequence length is 8K and the maximum batch size is constrained by memory capacity.

each providing 560 TFLOPS of BF16, 256GB of memory capacity, and 16TB/s of memory bandwidth.

Simulation: We model the AI accelerator equipped with the RoMe memory system, using LLMsimulator [77]. It allows configuring both the accelerator and the memory subsystem, supports continuous batching, and integrates Ramulator 2.0 [38] for cycle-accurate DRAM simulation. We implement RoMe in Ramulator 2.0, configuring both the accelerator and RoMe to process 4 KB requests. From the simulator, we collect time per output token (TPOT) and DRAM energy.

To ensure fair comparison, we sweep address mappings for both the baseline and RoMe, selecting the configuration that maximizes bandwidth utilization. We implement the MC for both systems using the FR-FCFS scheduling policy [60]. The baseline MC adopts an open-page policy, while both systems employ per-bank refresh commands to improve bandwidth availability. Table V summarizes the timing parameters used in our experiments. Because JEDEC has not finalized HBM4 timings, we adopt values from prior studies [2], [51].

LLM: We evaluate three large-scale LLMs: Grok 1 [73], DeepSeek-V3 [12], and Llama 3-405B (Llama 3 hereafter) [13]. DeepSeek-V3 uses Multi-head Latent Attention (MLA) and Mixture of Experts (MoE) together, Grok 1 adopts Grouped Query Attention (GQA) and MoE together, and Llama 3 adopts GQA but does not adopt MoE, instead using a fully-connected (FC) layer. For MoE, DeepSeek-V3 selects 8 of 256 experts per layer, while Grok 1 selects 2 of 8. All weights are stored in BF16.

During *prefill*, we apply tensor parallelism (TP) across the eight accelerators. During *decode*, TP is applied to the attention layers with degrees of 1, 8, and 8 for DeepSeek-V3, Grok 1, and Llama 3, respectively. It is because the compressed KV cache of MLA favors data parallelism to avoid TP communication overhead [78]. GQA runs with TP of 8, which our experiments and prior work have shown to be optimal [13]. For MoE, we use expert parallelism where each accelerator owns a distinct subset of experts, sending inputs to the target accelerator when a given expert is required and then receiving the output afterward.

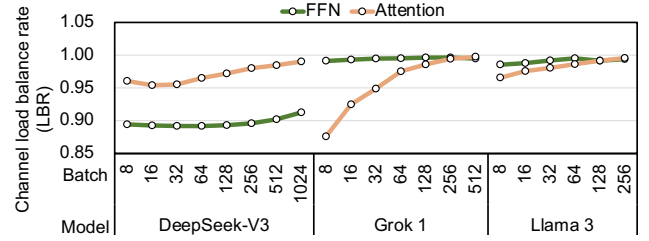


Fig. 13. Channel load balance ratio of RoMe across various batch sizes in DeepSeek-V3, Grok 1, and Llama 3 when the sequence length is 8K.

B. Performance Analysis of RoMe

We measured the TPOT of the baseline (HBM4) and RoMe during the *decode* stage with varying batch sizes when the sequence length is fixed at 8K. As shown in Figure 12, RoMe reduces TPOT by 10.4%, 10.2%, and 9.0% of HBM4 for DeepSeek-V3, Grok 1, and Llama 3, respectively. This improvement is largely attributed to RoMe's 12.5% higher memory bandwidth from its increased number of channels. However, the scaling does not fully align because several layers (*e.g.*, FFN layers) are not memory-bound.

Because RoMe operates at a 4 KB access granularity instead of the 32 B, load imbalance across memory channels becomes a critical concern for effective bandwidth utilization. Figure 13 shows channel load balance rate (*LBR*) of RoMe for attention (LBR_{Attn}) and FFN (LBR_{FFN}) layers across various batch sizes. *LBR* quantifies how uniformly data is distributed across memory channels, with its values normalized to the HBM4 baseline, whose *LBR* is nearly 1. The value closer to 1 indicates a more uniform data distribution across memory channels, enabling RoMe to fully utilize its available bandwidth, while lower values reflect increasing imbalance.

LBR differences across models primarily arise from their parallelization strategies and the relative contribution of weights and activations. In the attention layers, the hidden dimensions are 7,168 (DeepSeek-V3), 6,144 (Grok 1), and 16,384 (Llama 3), which are proportional to weight sizes. Given that data movement is dominated by weights at small batch sizes, DeepSeek-V3 adopts data parallelism, resulting in relatively high LBR_{Attn} even with a small KV-cache size due to MLA. In contrast, Grok 1 and Llama 3 employ TP and GQA, which reduces the data movement size of the weight per

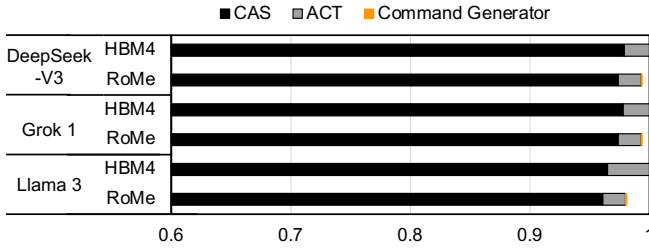


Fig. 14. Energy consumption of HBM4-based memory system and RoMe in DeepSeek-V3, Grok 1, and Llama 3 when the batch size is 256.

device, leading to lower LBR_{Attn} at small batches. However, Llama 3 still maintains high LBR_{Attn} because its large hidden dimension size keeps the weight contribution significant even under TP. As batch size increases, the KV-cache and activation footprints grow, improving LBR_{Attn} across all three models.

LBR_{FFN} is determined by their dimension size and architecture. DeepSeek-V3, with a small intermediate dimension of 2,048, shows relatively low LBR_{FFN} , while Grok 1 and Llama 3 have larger dimensions of 32,768 and 53,248, respectively. For the FFN layers, DeepSeek-V3 and Grok 1 employ an MoE architecture, while Llama 3 uses a dense architecture. In MoE layers, only a subset of experts (top-k) is activated, so LBR_{FFN} improves only at large batch sizes where more experts are selected. LBR_{FFN} improves once all experts begin to be selected, occurring around a batch of 64 in DeepSeek-V3 and a batch of 8 in Grok 1 in our experiments.

We omit results for the `prefill` stage, as its performance remains unchanged under both memory systems due to its compute-bound nature. This behavior stems from the characteristics of the `prefill` stage. Unlike the `decode` stage, which typically processes a single input token at a time, the `prefill` stage handles thousands of input tokens simultaneously. As a result, the workload is dominated by GEMM operations. Moreover, the large number of input tokens leads to significantly higher access volume to activations, weights, and KV-cache compared to the `decode` stage. Across all three evaluated LLMs, we observe that the performance difference in the `prefill` stage remains within 0.1%, confirming its insensitivity to the underlying memory system.

C. Area & Energy Overhead

First, we calculated the area overhead incurred by the four additional channels based on HBM3E specifications [34]. The μ bump pitch was assumed to be 22 μ m [62], which applies to both the DRAM and logic die. The number of μ bumps per channel was conservatively scaled by increasing it to four times the number required per channel [62], [77]. This configuration requires 48 additional μ bumps for additional TSVs, corresponding to an area of approximately 0.14 mm². Considering the edge margin, the DRAM die area increases by about 12%, and the logic die area grows proportionally, resulting in a total area overhead of only 0.10%.

We implemented the RoMe MC command scheduler and the command generator in Verilog, and utilized Synopsys Design Compiler with a 7nm process technology [9] to measure area

and energy consumption. Given that RoMe incorporates 36 legacy channels per cube, the total area overhead for the command generator amounts to 4268.8 μ m². This represents negligible overhead, occupying 0.003% of the logic die area.

For RoMe MC, we compared the area of the scheduling logic—including the command scheduler, bank FSM, and request queue—between RoMe and conventional MC. The request queue depth was set to 64 entries for the conventional MC and 4 entries for RoMe MC, both evaluated under the FR-FCFS scheduling policy [60]. Under these conditions, the command scheduling logic in RoMe MC occupies only 9.1% of the area of a conventional MC, indicating that RoMe achieves a much simpler architecture.

Figure 14 shows the energy consumption of LLM workloads under HBM4 and RoMe. We comprehensively calculated energy consumption by including the contributions of data movement within the HBM, command generator, and I/O interface. The underlying energy model for HBM4 is adopted from [2]. Compared to HBM4, RoMe reduces energy consumption by 1.9%, 0.7%, and 0.7% for the three evaluated LLMs, respectively. This improvement is primarily attributed to the decreased number of ACTs and the reduced energy consumption within the interposer. Specifically, the ACT energy consumption is reduced to 55.5%, 86.0%, and 84.4%, respectively. Because RoMe accesses DRAM via RD_row/WR_row, it requires only the minimal number of ACTs regardless of the amount of data accessed, thereby minimizing energy consumption from ACTs. Furthermore, interposer energy is reduced because RoMe MC issues a single RD_row or WR_row instead of 32 RDs or WRs. Although overfetch may slightly increase the number of RDs and WRs, the overall overhead remains marginal. Notably, the energy consumed by the command generator is negligible, contributing on average 0.06% relative to the total energy consumption. *Overall, these results indicate that RoMe achieves slight improvements in energy efficiency while providing noticeable performance gains.*

VII. DISCUSSION

Larger ECC codeword: RoMe may employ a larger ECC codeword by leveraging row access granularity (4 KB). HBM4 introduces two additional ECC pins per 32 DQ pins, building on the on-die ECC that has been available since HBM2E [61]. Using a larger ECC codeword can reduce parity-bit overhead while maintaining comparable error correction and detection capabilities [36], [74]. This approach expands the design space for ECC, enabling the use of either conventional ECC schemes or more robust low-overhead alternatives.

Hybrid architecture for fine-grained access: RoMe is optimized for LLMs with coarse access granularity, but performance may degrade under workloads with frequent fine-grained accesses, such as sparse attention in gpt-oss [54] and DeepSeek-V3.2 [11]. While gpt-oss maintains sequential access using sliding windows, DeepSeek Sparse Attention (DSA) selects top-2048 tokens from history, causing unpredictable and irregular access patterns when the sequence length exceeds 2048. This unpredictability can result in performance loss in

RoMe due to overfetch. To mitigate this, RoMe can adopt a heterogeneous system combining RoMe and conventional HBM4, assigning fine-grained requests to the latter. However, this may remain underutilized when processing fine-grained accesses, potentially leading to a reduction in overall bandwidth utilization. Another approach is enhancing RoMe’s memory controller and command generator to support selective column access via mask bits, though this introduces latency variation and added design complexity.

Processor-RoMe co-design: The effectiveness of RoMe critically depends on the processor’s computational unit. RoMe is optimized for row-granularity data movement, making it highly synergistic with processors that integrate a few large compute cores. A key example is the Google TPU, which employs a limited number of brawny cores (*e.g.*, 2 in TPUv7 [16]) with large shared on-chip buffers. Such a design naturally benefits from RoMe’s row access granularity by facilitating bulky and sequential data transfer. Moreover, systems supporting explicit data management to and from the on-chip memory are preferred. This programmability enables the processor to fully exploit RoMe’s row access mechanism, maximizing throughput while efficiently controlling data movement.

RoMe for training: RoMe can be applied seamlessly to LLM training workloads. Typically, LLM training operates on microbatches, where individual sequences are processed, each containing 8,192 tokens [39], [43]. During the training process, multiple tokens are processed in parallel to maximize computational throughput. This high degree of parallelism ensures that the memory access granularity is sufficiently large. Consequently, RoMe is highly feasible in training scenarios.

Other types of DRAM: The RoMe approach can be applied to other DRAM types [18] besides HBM. First, the simplification of the MC architecture remains applicable, allowing a reduction in the area overhead associated with scheduling. However, because only HBM incorporates a logic die, the placement of the command generator may differ, which could impose limitations on bandwidth expansion enabled by C/A pin reuse. In addition, unlike HBM, conventional DRAM devices have a limited number of connectable pins. Therefore, rather than aggregating the saved pins to construct additional channels as in RoMe, increasing the number of data pins could be a more effective approach for bandwidth expansion.

VIII. RELATED WORK

Coarse-grained access locality: Prior work has extensively documented the issue of memory access granularity arising from diverse application access patterns [1], [3], [36], [59], [74], [75], [81]. Specifically, [36], [59], [74], [75] identified problems regarding error correction, row buffer conflicts, and unnecessary data overfetch in systems that support a static granularity. They proposed memory systems that support adaptive memory access granularity for different applications. However, these designs do not take the memory access patterns specific to LLMs into account. Further, because they keep the conventional memory interface unchanged, they still incur complex scheduling overhead.

Fine-grained access granularity: A body of work [2], [4], [7], [17], [35], [53], [64], [65], [69], [79], [80], [83] has proposed energy-efficient fine-grained DRAM architectures. [4], [83] enable DRAM chips within a DRAM module, which originally operate in lock-step, to function independently. Fine-grained DRAM architectures [7], [17], [35], [51], [64], [69], [79], [80] divide the DRAM bank into independently operating fine-grained memory arrays. They primarily lower row activation energy by reducing the row size or saving access energy from data overfetch through finer access granularity, especially for applications with low spatial locality. In contrast to prior work, RoMe focuses on the highly sequential memory access pattern of LLM inference, which both minimizes the overfetch overhead and fully exploits channel- and bank-level parallelism. Through this approach, RoMe achieves a simplified memory interface, greater scalability, improved performance, and higher energy efficiency—even while increasing the row size and access granularity.

IX. CONCLUSION

Motivated by the sequential and large memory traffic exhibited by large language models (LLMs), this paper proposes RoMe, a DRAM subsystem that adopts a row-granularity interface. RoMe supports only row-level read and write commands, eliminating column, bank group, and pseudo channel layers required by conventional HBM and dispensing with cache line transfers entirely. Freed from cache line granularity, we introduce the VBA organization, which efficiently supports row-level access. We also present a command generator on the logic die that reduces the C/A pin requirements between the memory controller and HBM. The freed pins are repurposed as additional channel interfaces, enabling efficient HBM channel expansion with minimal cost. By simplifying the HBM hierarchy, RoMe lightens both the scheduling logic and hardware overhead of the memory controller. Experiments on representative LLM workloads demonstrate that RoMe achieves higher performance and energy efficiency than HBM4 with minimal additional hardware overhead. Any overfetch or load imbalance side effects introduced by row-granularity access remain negligible.

ACKNOWLEDGEMENTS

We appreciate the input from Gunjun Lee at Seoul National University (SNU) regarding sparse attention. This research was in part supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) [RS-2021-II211343, RS-2024-00456287, RS-2024-00402898, RS-2025-02304125], and by the National Research Foundation of Korea (NRF) grant funded by MSIT [RS-2024-00405857]. The EDA tool was supported by the IC Design Education Center (IDEC), Korea. This work was done when Michael Jaemin Kim was at SNU. Jung Ho Ahn, the corresponding author, is with the Department of Intelligence and Information and the Interdisciplinary Program in Artificial Intelligence, SNU.

REFERENCES

- [1] A. Abulila, V. S. Malthody, Z. Qureshi, J. Huang, N. S. Kim, J. Xiong, and W.-m. Hwu, "FlatFlash: Exploiting the Byte-Accessibility of SSDs within a Unified Memory-Storage Hierarchy," in *ASPLOS*, 2019.
- [2] V. Adhinarayanan, B. M. Beckmann, W. Li, M. Seyedzadeh, S. Blagoderov, D. Aguren, and H. H. Lee, "Folded Banks: 3D-Stacked HBM Design for Fine-Grained Random-Access Bandwidth," in *ISCA*, 2025.
- [3] J. Ahn, M. Erez, and W. J. Dally, "The Design Space of Data-Parallel Memory Systems," in *SC*, 2006.
- [4] J. Ahn, N. P. Jouppi, C. Kozyrakis, J. Leverich, and R. S. Schreiber, "Future Scaling of Processor-Memory Interfaces," in *SC*, 2009.
- [5] R. Ausavarungrun, K. K.-W. Chang, L. Subramanian, G. H. Loh, and O. Mutlu, "Staged Memory Scheduling: Achieving High Performance and Scalability in Heterogeneous Systems," in *ISCA*, 2012.
- [6] S. Baek, M. Wi, S. Park, H. Nam, M. J. Kim, N. S. Kim, and J. Ahn, "Marionette: A RowHammer Attack via Row Coupling," in *ASPLOS*, 2025.
- [7] N. Chatterjee, M. O'Connor, D. Lee, D. R. Johnson, S. W. Keckler, M. Rhu, and W. J. Dally, "Architecting an Energy-Efficient DRAM System for GPUs," in *HPCA*, 2017.
- [8] J. H. Cho, J. Kim, W. Y. Lee, D. U. Lee, T. K. Kim, H. B. Park, C. Jeong, M.-J. Park, S. G. Baek, S. Choi, B. K. Yoon, Y. J. Choi, K. Y. Lee, D. Shim, J. Oh, J. Kim, and S.-H. Lee, "A 1.2V 64Gb 341GB/s HBM2 Stacked DRAM with Spiral Point-to-Point TSV Structure and Improved Bank Group Data Control," in *2018 IEEE International Solid-State Circuits Conference (ISSCC)*, 2018.
- [9] L. T. Clark, V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric, "ASAP7: A 7-nm finFET Predictive Process Design Kit," *Microelectronics Journal*, 2016. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S002626921630026X>
- [10] W. J. Dally and B. P. Towles, *Principles and Practices of Interconnection Networks*. Morgan Kaufmann Publishers Inc., 2004.
- [11] DeepSeek-AI, "DeepSeek-V3.2-Exp: Boosting Long-Context Efficiency with DeepSeek Sparse Attention," 2025. [Online]. Available: <https://github.com/deepseek-ai/DeepSeek-V3.2-Exp>
- [12] DeepSeek-AI, A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Wang, J. Chen, J. Chen, J. Yuan, J. Qiu, J. Li, J. Song, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Wang, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Wang, Q. Zhu, Q. Chen, Q. Du, R. Chen, R. Jin, R. Ge, R. Zhang, R. Pan, R. Wang, R. Xu, R. Zhang, R. Chen, S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Pan, T. Wang, T. Yun, T. Pei, T. Sun, W. Xiao, W. Zeng, W. Zhao, W. An, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, X. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Zhang, X. Chen, X. Nie, X. Sun, X. Wang, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yu, X. Song, X. Shan, X. Zhou, X. Yang, X. Li, X. Su, X. Lin, Y. Li, Y. Wang, Y. Wei, Y. Zhu, Y. Zhang, Y. Xu, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Yu, Y. Zheng, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Tang, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Wu, Y. Ou, Y. Zhu, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Zha, Y. Xiong, Y. Ma, Y. Yan, Y. Luo, Y. You, Y. Liu, Y. Zhou, Z. Wu, Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Huang, Z. Zhang, Z. Xie, Z. Zhang, Z. Hao, Z. Gou, Z. Ma, Z. Yan, Z. Shao, Z. Xu, Z. Wu, Z. Zhang, Z. Li, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Gao, and Z. Pan, "DeepSeek-V3 Technical Report," 2024. [Online]. Available: <https://arxiv.org/abs/2412.19437>
- [13] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Srivankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. Ferrer, Canton, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. Smith, Michael, F. Radenovic, F. Zhang, G. Synnaeve, G. Lee, G. Anderson, Lewis, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. Ibarra, Arrieta, I. Kloumann, I. Misra, I. Evtimov, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. v. d. Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bittin, J. Spisak, J. Park, J. Rocca, J. Johnstun, J. Saxe, J. Jia, K. Alwala, Vasuden, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, L. Rantala-Yeary, L. v. d. Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. Oliveira, de, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. Singh, Kumar, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. Koura, Singh, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. Cabral, Silveira, R. Stojnic, R. Raileanu, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. Kim, Sonia, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenheide, S. Batra, S. Whitman, S. Sootla, S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. Tan, Ellen, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z. Coudert, Delpierre, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Grattafiori, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Goldstand, A. Menon, A. Sharma, A. Boesenberg, A. Vaughan, A. Baevski, A. Feinstein, A. Kallet, A. Sangani, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Franco, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. Paola, De, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, D. Civin, D. Beaty, D. Kreymer, D. Li, D. Wyatt, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Ozgenel, F. Caggioni, F. Guzmán, F. Kanayet, F. Seide, G. Florez, Medina, G. Schwarz, G. Badeer, G. Swee, G. Halpern, G. Thattai, G. Herman, G. Sizov, Guangyi, Zhang, G. Lakshminarayanan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, I. Damaj, I. Molybog, I. Tufanov, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, K. U. Hou, K. Saxena, K. Prasad, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelenia, K. Li, K. Huang, K. Chawla, K. Lakhotia, K. Huang, L. Chen, L. Garg, L. A. L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabsa, M. Avalani, M. Bhatt, M. Tsimpoukelli, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. Seltzer, L., M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. Hermoso, Jubert, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. Laptev, Pavlovich, N. Dong, N. Zhang, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dollar, P. Zvyagina, P. Ratanachandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Li, R. Hogan, R. Battery, R. Wang, R. Maheswari, R. Howes, R. Rinott, S. Bondu, Jayesh, S. Datta, S. Chugh, S. Hunt, S. Dhillon, S. Sidorov, S. Pan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S. Zha, Cindy, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Kohler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. Kumar, Satish, V. Mangla, V. Albiero, V. Ionescu, V. Poenaru, V. Mihailescu, Tiberiu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wang, X. Wu, X. Wang, X. Xia, X. Wu,

- X. Gao, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Yu. Wang, Y. Hao, Y. Qian, Y. He, Z. Rait, Z. DeVito, Z. Rosnbrick, Z. Wen, Z. Yang, and Z. Zhao, "The Llama 3 Herd of Models," 2024.
- [14] X. Fu, Z. Zhang, H. Fan, G. Huang, M. El-Shabani, R. Huang, R. Solanki, F. Wu, R. Diamant, and Y. Wang, "Distributed Training of Large Language Models on AWS Trainium," in *Proceedings of the 2024 ACM Symposium on Cloud Computing (SoCC)*, 2024.
- [15] B. Fujun, J. Xiping, W. Song, Y. Bing, T. Jie, Z. Fengguo, W. Chunjuan, W. Fan, L. Xiaodong, Y. Guoqing, F. Ni, L. Qiannan, L. Hua, W. Kexin, D. Huifu, B. Liang, J. Xuerong, L. Jin, L. Mei, W. Zhengwen, H. Sheng, Z. Jun, Z. Qiong, S. Peng, Y. Daohong, C. Kau, D. Yang, C.-S. Ho, S. Hongbin, L. Hangbing, L. Ming, K. Yi, and R. Qiwei, "A Stacked Embedded DRAM Array for LPDDR4/4X using Hybrid Bonding 3D Integration with 34GB/s/1Gb 0.88pJ/b Logic-to-Memory Interface," in *IEEE International Electron Devices Meeting (IEDM)*, 2020.
- [16] Google, "Ironwood: The First Google TPU for the Age of Inference," 2025. [Online]. Available: <https://blog.google/products/google-cloud/ironwood-tpu-age-of-inference/>
- [17] H. Ha, A. Pedram, S. Richardson, S. Kvatinsky, and M. Horowitz, "Improving Energy Efficiency of DRAM by Exploiting Half Page Row Access," in *MICRO*, 2016.
- [18] H. Hassan, A. Olgun, A. G. Yağlıkçı, H. Luo, O. Mutlu, and E. Zurich, "Self-Managing DRAM: A Low-Cost Framework for Enabling Autonomous and Efficient DRAM Maintenance Operations," in *MICRO*, 2024.
- [19] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing," in *ASPLOS*, 2024.
- [20] Intel, "Performance Differences for Open-Page/Close-Page Policy," <https://www.intel.com/content/www/us/en/content-details/826015/performance-differences-for-open-page-close-page-policy.html>, 2024.
- [21] P. Jattke, M. Wipfli, F. Solt, M. Marazzi, M. Bölskei, and K. Razavi, "ZenHammer: Rowhammer Attacks on AMD Zen-based Platforms," in *USENIX Security Symposium*, 2024.
- [22] JEDEC, "High Bandwidth Memory (HBM) DRAM," 2013.
- [23] JEDEC, "DDR4 SDRAM Standard," 2017.
- [24] JEDEC, "High Bandwidth Memory DRAM (HBM1, HBM2) Standard," 2018.
- [25] JEDEC, "High Bandwidth Memory DRAM (HBM3) Standard," 2022.
- [26] JEDEC, "DDR5 SDRAM Standard," 2024.
- [27] JEDEC, "High Bandwidth Memory (HBM4) DRAM," 2025.
- [28] N. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles, C. Young, X. Zhou, Z. Zhou, and D. A. Patterson, "TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings," in *ISCA*, 2023.
- [29] D. Kaseridis, J. Stuecheli, and L. K. John, "Minimalist Open-page: A DRAM Page-mode Scheduling Policy for the Many-core Era," in *MICRO*, 2011.
- [30] H. Kim, Y. Choi, J. Park, B. Bae, H. Jeong, S. M. Lee, J. Yeon, M. Kim, C. Park, B. Gu, C. Lee, J. Bae, S. Bae, Y. Cha, W. Choe, J. Choi, J. Ha, H. Han, N. Hwang, S. Hwang, K. Jang, H. Je, H. Jeon, J. Jeon, H. Jeong, Y. Jung, D. Kang, H. Kim, M. Kim, M. Kim, S. Kim, S. Kim, W. Kim, Y. Kim, Y. Kim, Y. Ku, J. K. Lee, J. Lee, K. Lee, S. Lee, M. Noh, H. Oh, G. Park, S. Park, J. Seo, J. Seong, J. Paik, N. P. Lopes, and S. Yoo, "TCP: A Tensor Contraction Processor for AI Workloads," in *ISCA*, 2024.
- [31] Y. Kim, D. Han, O. Mutlu, and M. Harchol-Balter, "ATLAS: A Scalable and High-Performance Scheduling Algorithm for Multiple Memory Controllers," in *HPCA*, 2010.
- [32] Y. Kim, M. Papamichael, O. Mutlu, and M. Harchol-Balter, "Thread Cluster Memory Scheduling: Exploiting Differences in Memory Access Behavior," in *HPCA*, 2010.
- [33] D. U. Lee, K. W. Kim, K. W. Kim, H. Kim, J. Y. Kim, Y. J. Park, J. H. Kim, D. S. Kim, H. B. Park, J. W. Shin, J. H. Cho, K. H. Kwon, M. J. Kim, J. Lee, K. W. Park, B. Chung, and S. Hong, "25.2 A 1.2V 8Gb 8-Channel 128GB/s High-Bandwidth Memory (HBM) Stacked DRAM with Effective Microbump I/O Test Methods Using 29nm Process and TSV," in *IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, 2014.
- [34] J. Lee, K. Cho, C. K. Lee, Y. Lee, J.-H. Park, S.-H. Oh, Y. Ju, C. Jeong, H. S. Cho, J. Lee, T.-S. Yun, J. H. Cho, S. Oh, J. Moon, Y.-J. Park, H.-S. Choi, I.-K. Kim, S. M. Yang, S.-Y. Kim, J. Jang, J. Kim, S.-H. Lee, Y. Jeon, J. Park, T.-K. Kim, D. Ka, S. Oh, J. Kim, J. Jeon, S. Kim, K. T. Kim, T. Kim, H. Yang, D. Yang, M. Lee, H. Song, D. Jang, J. Shin, H. Kim, C. Baek, H. Jeong, J. Yoon, S.-K. Lim, K. Y. Lee, Y. J. Koo, M.-J. Park, J. Cho, and J. Kim, "13.4 A 48GB 16-High 1280GB/s HBM3E DRAM with All-Around Power TSV and a 6-Phase RDQS Scheme for TSV Area Optimization," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2024.
- [35] Y. Lee, H. Kim, S. Hong, and S. Kim, "Partial Row Activation for Low-Power DRAM System," in *HPCA*, 2017.
- [36] S. Li, D. H. Yoon, K. Chen, J. Zhao, J. Ahn, J. B. Brockman, Y. Xie, and N. P. Jouppi, "MAGE: Adaptive Granularity and ECC for Resilient and Power Efficient Memory Systems," in *SC*, 2012.
- [37] A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Dengr, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Yang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Chen, J. Yuan, J. Qiu, J. Song, K. Dong, K. Gao, K. Guan, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Zhu, Q. Chen, Q. Du, R. Chen, R. Jin, R. Ge, R. Pan, R. Xu, R. Chen, S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Zheng, T. Wang, T. Pei, T. Yuan, T. Sun, W. Xiao, W. Zeng, W. An, W. Liu, W. Liang, W. Gao, W. Zhang, X. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Chen, X. Nie, X. Sun, X. Wang, X. Liu, X. Xie, X. Yu, X. Song, X. Zhou, X. Yang, X. Lu, X. Su, Y. Wu, Y. Li, Y. Wei, Y. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Huang, Y. Zheng, Y. Zhang, Y. Xiong, Y. Zhao, Y. He, Y. Tang, Y. Piao, Y. Dong, Y. Tan, Y. Liu, Y. Wang, Y. Guo, Y. Zhu, Y. Wang, Y. Zou, Y. Zha, Y. Ma, Y. Yan, Y. You, Y. Liu, Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Huang, Z. Zhang, Z. Xie, Z. Hao, Z. Shao, Z. Wen, Z. Xu, Z. Zhang, Z. Li, Z. Wang, Z. Gu, Z. Li, and Z. Xie, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," 2024. [Online]. Available: <https://arxiv.org/abs/2405.04434>
- [38] H. Luo, Y. C. Tuğrul, F. N. Bostancı, A. Olgun, A. G. Yağlıkçı, and O. Mutlu, "Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator," 2023. [Online]. Available: <https://github.com/CMU-SAFARI/ramulator2.git>
- [39] MLPerf, "Training Policies," accessed in Oct 2025. [Online]. Available: https://github.com/mlcommons/training_policies/tree/master
- [40] O. Mutlu and T. Moscibroda, "Stall-Time Fair Memory Access Scheduling for Chip Multiprocessors," in *MICRO*, 2007, pp. 146–160.
- [41] O. Mutlu and T. Moscibroda, "Parallelism-Aware Batch Scheduling: Enhancing both Performance and Fairness of Shared DRAM Systems," in *ISCA*, 2008.
- [42] H. Nam, S. Baek, M. Wi, M. J. Kim, J. Park, C. Song, N. S. Kim, and J. Ahn, "DRAMScope: Uncovering DRAM Microarchitecture and Characteristics by Issuing Memory Commands," in *ISCA*, 2024.
- [43] D. Narayanan, M. Shoenybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia, "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," in *SC*, 2021.
- [44] K. J. Nesbit, N. Aggarwal, J. Laudon, and J. E. Smith, "Fair Queuing Memory Systems," in *MICRO*, 2006, pp. 208–222.
- [45] D. Niu, S. Li, Y. Wang, W. Han, Z. Zhang, Y. Guan, T. Guan, F. Sun, F. Xue, L. Duan, Y. Fang, H. Zheng, X. Jiang, S. Wang, F. Zuo, Y. Wang, B. Yu, Q. Ren, and Y. Xie, "184QPS/W 64Mb/mm23D Logic-to-DRAM Hybrid Bonding with Process-Near-Memory Engine for Recommendation System," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2022.
- [46] NVIDIA, "NVIDIA A100 Tensor Core GPU Architecture," 2020. [Online]. Available: <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf>
- [47] NVIDIA, "NVIDIA H100 GPU," 2024. [Online]. Available: <https://resources.nvidia.com/en-us-hopper-architecture/nvidia-tensor-core-gpu-datasheet>
- [48] NVIDIA, "NVIDIA Blackwell Architecture Technical Brief," 2025. [Online]. Available: <https://resources.nvidia.com/en-us-blackwell-architecture>
- [49] NVIDIA, "NVIDIA Blackwell Architecture Technical Brief," 2025. [Online]. Available: <https://www.nvidia.com/en-us/solutions/ai-factories>
- [50] S. O, Y. H. Son, N. S. Kim, and J. Ahn, "Row-buffer decoupling: A case for low-latency dram microarchitecture," in *ISCA*, 2014.

- [51] M. O'Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, "Fine-Grained DRAM: Energy-Efficient DRAM for Extreme Bandwidth Systems," in *MICRO*, 2017.
- [52] C.-S. Oh, K. C. Chun, Y.-Y. Byun, Y.-K. Kim, S.-Y. Kim, Y. Ryu, J. Park, S. Kim, S. Cha, D. Shin, J. Lee, J.-P. Son, B.-K. Ho, S.-J. Cho, B. Kil, S. Ahn, B. Lim, Y. Park, K. Lee, M.-K. Lee, S. Baek, J. Noh, J.-W. Lee, S. Lee, S. Kim, B. Lim, S.-K. Choi, J.-G. Kim, H.-I. Choi, H.-J. Kwon, J. J. Kong, K. Sohn, N. S. Kim, K.-I. Park, and J.-B. Lee, "22.1 A 1.1V 16GB 640GB/s HBM2E DRAM with a Data-Bus Window-Extension Technique and a Synergetic On-Die ECC Scheme," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2020.
- [53] A. Olgun, F. N. Bostanci, G. Francisco de Oliveira Junior, Y. C. Tugrul, R. Bera, A. G. Yaglikci, H. Hassan, O. Ergin, and O. Mutlu, "Sectored DRAM: A Practical Energy-Efficient and High-Performance Fine-Grained DRAM Architecture," *TACO*, 2024.
- [54] OpenAI, "gpt-oss-120b & gpt-oss-20b Model Card," 2025. [Online]. Available: <https://arxiv.org/abs/2508.10925>
- [55] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. Ahn, "AttAcc: Unleashing the Power of PIM for Batched Transformer-based Generative Model Inference," in *ASPLOS*, 2024.
- [56] M.-J. Park, H. S. Cho, T.-S. Yun, S. Byeon, Y. J. Koo, S. Yoon, D. U. Lee, S. Choi, J. Park, J. Lee, K. Cho, J. Moon, B.-K. Yoon, Y.-J. Park, S.-m. Oh, C. K. Lee, T.-K. Kim, S.-H. Lee, H.-W. Kim, Y. Ju, S.-K. Lim, S. G. Baek, K. Y. Lee, S. H. Lee, W. S. We, S. Kim, Y. Choi, S.-H. Lee, S. M. Yang, G. Lee, I.-K. Kim, Y. Jeon, J.-H. Park, J. C. Yun, C. Park, S.-Y. Kim, S. Kim, D.-Y. Lee, S.-H. Oh, T. Hwang, J. Shin, Y. Lee, H. Kim, J. Lee, Y. Hur, S. Lee, J. Jang, J. Chun, and J. Cho, "A 192-Gb 12-High 896-GB/s HBM3 DRAM with a TSV Auto-Calibration Scheme and Machine-Learning-Based Layout Optimization," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2022.
- [57] P. Patel, E. Choukse, C. Zhang, A. Shah, Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient Generative LLM Inference Using Phase Splitting," in *ISCA*, 2024.
- [58] P. Pessl, D. Gruss, C. Maurice, M. Schwarz, and S. Mangard, "DRAMA: Exploiting DRAM Addressing for Cross-CPU Attacks," in *25th USENIX Security Symposium (USENIX Security 16)*, 2016.
- [59] M. Rhu, M. Sullivan, J. Leng, and M. Erez, "A Locality-Aware Memory Hierarchy for Energy-Efficient GPU Architectures," in *MICRO*, 2013.
- [60] S. Rixner, W. J. Dally, U. J. Kapasi, P. Mattson, and J. D. Owens, "Memory Access Scheduling," in *ISCA*, 2000.
- [61] Y. Ryu, S.-G. Ahn, J. H. Lee, J. Park, Y. K. Kim, H. Kim, Y. G. Song, H.-W. Cho, S. Cho, S. H. Song, H. Lee, U. Shin, J. Ahn, J.-M. Ryu, S. Lee, K.-H. Lim, J. Lee, J. H. Park, J.-S. Jeong, S. Joo, D. Cho, S. Y. Kim, M. Lee, H. Kim, M. Kim, J.-S. Kim, J. Kim, H. G. Kang, M.-K. Lee, S.-R. Kim, Y.-C. Kwon, Y. Y. Byun, K. Lee, S. Park, J. Youn, M.-O. Kim, K. Sohn, S.-J. Hwang, and J. Lee, "A 16 GB 1024 GB/s HBM3 DRAM With Source-Synchronized Bus Design and On-Die Error Control Scheme for Enhanced RAS Features," *IEEE Journal of Solid-State Circuits*, 2023.
- [62] SK hynix, "Advanced Packaging Technology for Beyond Memory," 2023. [Online]. Available: https://www.theise.org/wp-content/uploads/2023/10/Tutorial1-4_%EC%86%90%ED%98%B8%EC%98%81%EC%88%98%EC%84%9D%EB%8B%98_SK%ED%95%98%EC%9D%B4%EB%8B%89%EC%8A%A4.pdf
- [63] A. Smith, G. H. Loh, M. J. Schulte, M. Ignatowski, S. Naffziger, M. Mantor, M. Fowler, N. Kalyanasundharam, V. Alla, N. Malaya, J. L. Greathouse, E. Chapman, and R. Swaminathan, "Realizing the AMD Exascale Heterogeneous Processor Vision," in *ISCA*, 2024.
- [64] Y. H. Son, O. Seongil, H. Yang, D. Jung, J. Ahn, J. Kim, J. Kim, and J. W. Lee, "Microbank: Architecting Through-Silicon Interposer-Based Main Memory Systems," in *SC*, 2014.
- [65] K. Sudan, N. Chatterjee, D. Nellans, M. Awasthi, R. Balasubramonian, and A. Davis, "Micro-pages: Increasing DRAM Efficiency with Locality-aware Data Placement," in *ASPLOS*, 2010.
- [66] The Korea Economic Daily Global Edition, "SK Hynix Ships World's First 12-Layer HBM4 Samples Early," 2025. [Online]. Available: <https://www.kedglobal.com/korean-chipmakers/newsView/ked202503190006>
- [67] The Korean Economic Daily, "Samsung to Mass-Produce HBM4 on 4 nm Foundry Process," 2024. [Online]. Available: <https://www.kedglobal.com/korean-chipmakers/newsView/ked202407150016>
- [68] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open Foundation and Fine-Tuned Chat Models," 2023. [Online]. Available: <https://arxiv.org/abs/2307.09288>
- [69] A. N. Udipi, N. Muralimanohar, N. Chatterjee, R. Balasubramonian, A. Davis, and N. P. Jouppi, "Rethinking DRAM Design and Organization for Energy-Constrained Multi-Cores," in *ISCA*, 2010.
- [70] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is All You Need," in *NeurIPS*, 2017.
- [71] M. Wang, Z. Zhang, Y. Cheng, and S. Nepal, "DRAMDig: A Knowledge-assisted Tool to Uncover DRAM Address Mapping," in *Design Automation Conference (DAC)*, 2020.
- [72] M. Wi, S. Baek, S. Park, M. Erez, and J. H. Ahn, "Sudoku: Decomposing DRAM Address Mapping into Component Functions," *arXiv preprint arXiv:2506.15918*, 2025.
- [73] xAI, "grok1," 2024. [Online]. Available: <https://github.com/xai-org/grok-1>
- [74] D. H. Yoon, M. K. Jeong, and M. Erez, "Adaptive Granularity Memory Systems: A Tradeoff between Storage Efficiency and Throughput," in *ISCA*, 2011.
- [75] D. H. Yoon, M. K. Jeong, M. Sullivan, and M. Erez, "The Dynamic Granularity Memory System," in *ISCA*, 2012.
- [76] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A Distributed Serving System for Transformer-Based Generative Models," in *USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022.
- [77] S. Yun, K. Kyung, J. Cho, J. Choi, J. Kim, B. Kim, S. Lee, K. Sohn, and J. Ahn, "Duplex: A Device for Large Language Models with Mixture of Experts, Grouped Query Attention, and Continuous Batching," in *MICRO*, 2024.
- [78] S. Yun, S. Park, H. Nam, Y. Lee, G. Lee, K. Kyung, S. Kim, N. S. Kim, J. Kim, H. Kim, J. Cho, S. Baek, and J. Ahn, "The New LLM Bottleneck: A Systems Perspective on Latent Attention and Mixture-of-Experts," 2025. [Online]. Available: <https://arxiv.org/abs/2507.15465>
- [79] C. Zhang and X. Guo, "Enabling Efficient Fine-Grained DRAM Activations with Interleaved I/O," in *ISLPED*, 2017.
- [80] T. Zhang, K. Chen, C. Xu, G. Sun, T. Wang, and Y. Xie, "Half-DRAM: A High-bandwidth and Low-power DRAM Architecture from the Rethinking of Fine-grained Activation," in *ISCA*, 2014.
- [81] X. Zhang, T. Lu, Y. Chang, K. Zhang, and M. Chen, "Morpheus: An Adaptive DRAM Cache with Online Granularity Adjustment for Disaggregated Memory," in *International Conference on Computer Design (ICCD)*, 2023.
- [82] Z. Zhang, Z. Zhu, and X. Zhang, "A Permutation-based Page Interleaving Scheme to Reduce Row-buffer Conflicts and Exploit Data Locality," in *MICRO*, 2000.
- [83] H. Zheng, J. Lin, Z. Zhang, E. Gorbato, H. David, and Z. Zhu, "Mini-Rank: Adaptive DRAM Architecture for Improving Memory Power Efficiency," in *MICRO*, 2008.